МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Инженерно-экономический факультет
Кафедра экономической информатики

Отчет
по лабораторной работе №4

на тему «Управление персистентностью на основе JPA (Hibernate)»
по курсу «Современные технологии серверной разработки»

Выполнил: Р. В. Тамашевский
472303

Проверила: Е. А. Мельникова

Минск 2026

Цель: изучить спецификацию JPA и её реализацию Hibernate, приобрести навыки отображения классов Java на таблицы реляционной базы данных, настройки связей между сущностями и выполнения запросов на языке HQL и с помощью JPA Criteria API.

Задание: Изучить конфигурирование Hibernate на основе XML и аннотаций, язык запросов HQL, JPA Criteria, отношения между таблицами, проблему n+1 и типы извлечения данных. В проекте раздела «Отношения между таблицами» дописать код в местах, помеченных как задание: поиск записи по идентификатору, добавление записи, перевод связанной записи к другой родительской записи, удаление записи вместе со связанными и поиск пользователя по имени. Написать три собственных SELECT-запроса с использованием HQL или Criteria. Работа выполняется для предметной области курсового проекта «Халтура» – веб-сервиса взаимодействия заказчиков и исполнителей, в качестве СУБД используется PostgreSQL.

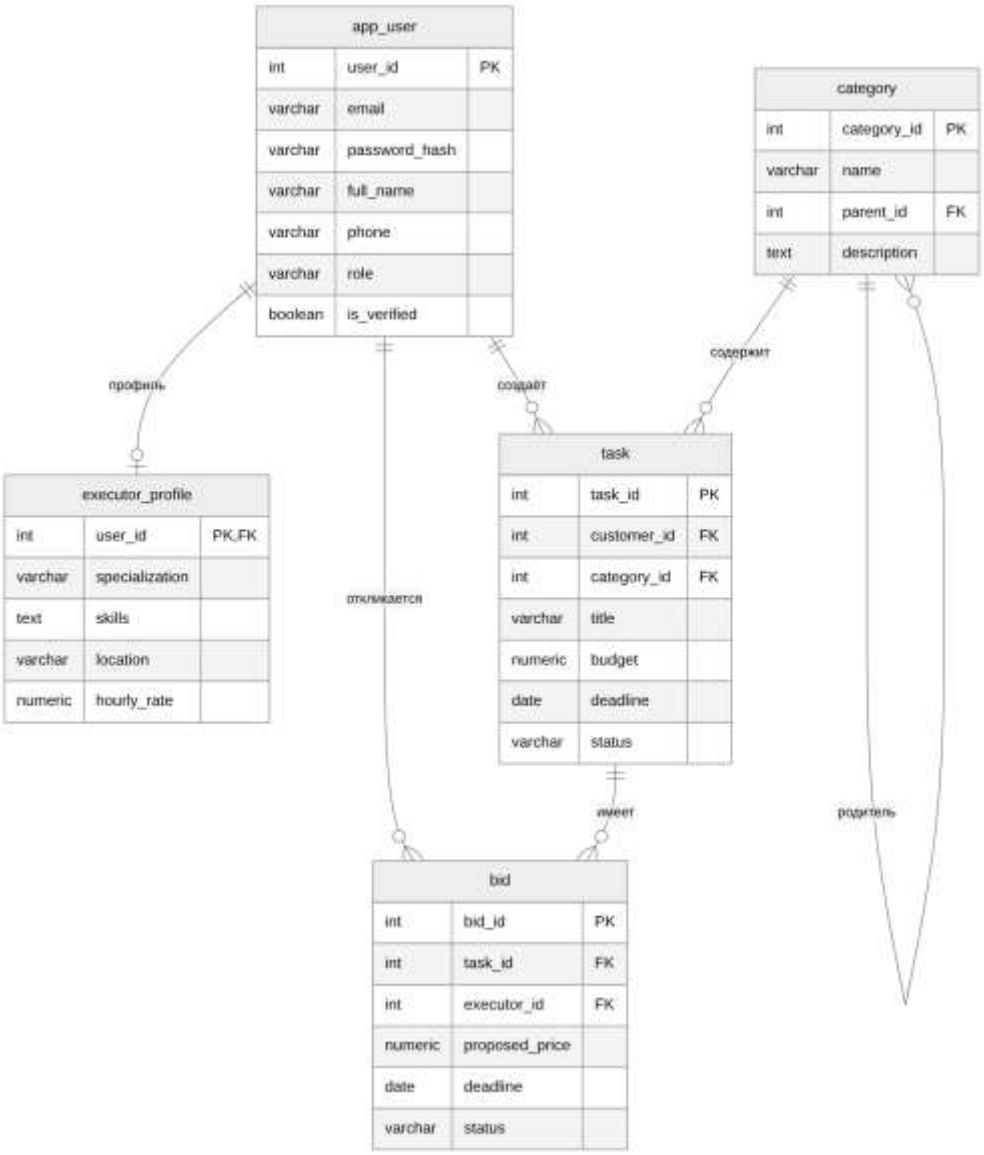
Ход выполнения работы:

Предметная область – сервис заказа бытовых и профессиональных услуг. Заказчик создаёт задачу (Task) в одной из категорий (Category) с бюджетом, сроком и адресом. После проверки администратором задача публикуется, и исполнители оставляют на неё отклики (Bid) с предложенной ценой, сроком и текстом. Заказчик выбирает один из откликов, после чего задача переходит в работу. У исполнителя есть профиль (ExecutorProfile) со специализацией, навыками, городом и ставкой. Пользователь (AppUser) работает в системе в одной из ролей: заказчик, исполнитель, заказчик и исполнитель, администратор.

Сущности методического примера сопоставлены с предметной областью так: отдел и сотрудники заменены задачей и откликами (связь один ко многим с каскадным удалением), сотрудник и учётная запись – пользователем и профилем исполнителя (связь один к одному с общим первичным ключом).

1 База данных

Создадим базу данных Modern-server-side-development-technologies-LW4. Создаём пять таблиц: app_user (пользователи), executor_profile (профили исполнителей), category (категории), task (задачи) и bid (отклики), а также заполняет их демонстрационными данными. Схема базы данных:



Целостность данных обеспечивается на уровне СУБД. Допустимые роли и статусы задаются ограничениями CHECK, ограничение UNIQUE (task_id, executor_id) не позволяет исполнителю откликнуться на одну задачу дважды, а внешний ключ bid.task_id объявлен с ON DELETE CASCADE. Первичный ключ executor_profile одновременно является внешним ключом на app_user, что реализует связь один к одному. Категории образуют дерево: поле parent_id ссылается на ту же таблицу. Пароли пользователей хранятся в виде bcrypt-хэша. Поскольку файл скрипта сохранён в UTF-8, а консоль Windows использует другую кодировку, в начале скрипта выполняется команда SET client_encoding = 'UTF8'.

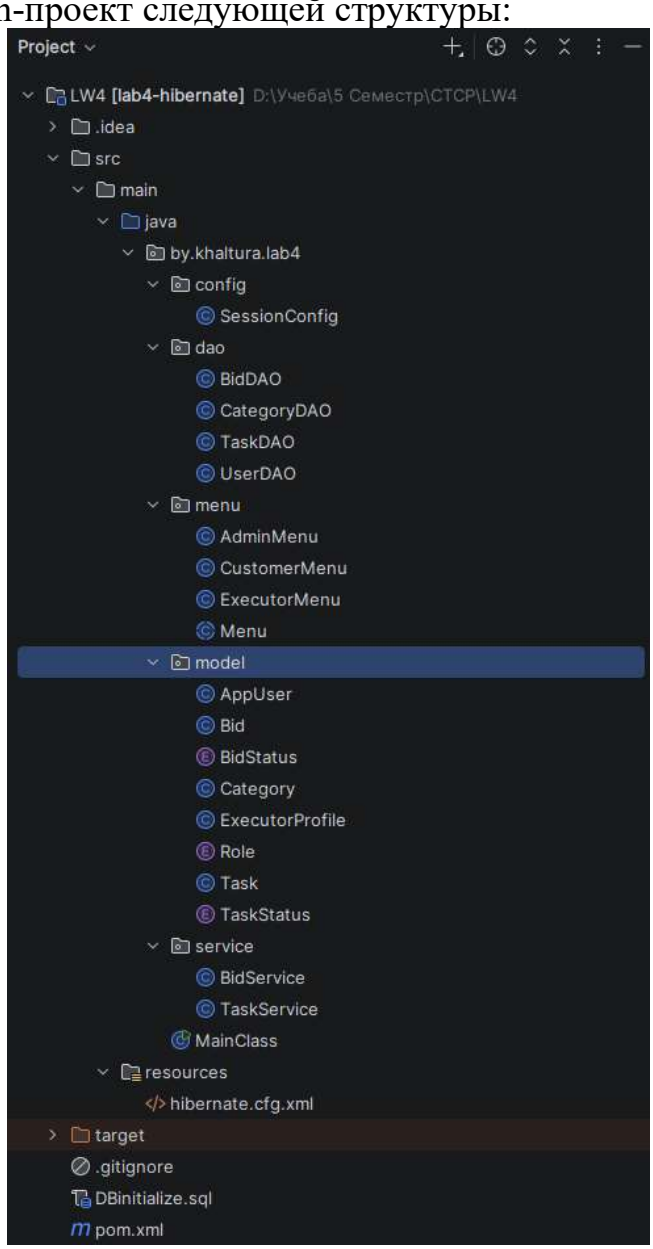
Результат выполнения скрипта:

```
D:\Учеба\5 Семестр\CTCP\LW4> psql -U postgres -d Modern-server-side-development-technologies-LW4 -f DBinitialize.sql
Password for user postgres:
SET
DROP TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE INDEX
CREATE INDEX
INSERT 0 6
INSERT 0 3
INSERT 0 5
INSERT 0 2
INSERT 0 5
INSERT 0 5

D:\Учеба\5 Семестр\CTCP\LW4>
```

2 Структура и конфигурация проекта

Создадим Maven-проект следующей структуры:



В pom-файл добавлены зависимости hibernate-core 6.6.1.Final (реализация JPA), postgresql 42.7.4 (JDBC-драйвер PostgreSQL) и jbcrypt (хэширование паролей).

В файле hibernate.cfg.xml указаны параметры подключения к базе данных и свойство current_session_context_class = thread, благодаря которому метод getCurrentSession() возвращает сессию, привязанную к текущему потоку. Свойство hbm2ddl.auto = validate заставляет Hibernate при запуске сверять классы-модели со структурой таблиц: при расхождении приложение не запустится и сообщит об ошибке. Классы-модели в конфигурационном файле не перечисляются – они регистрируются программно.

Класс SessionConfig реализует шаблон «Одиночка»: создаёт единственную фабрику сессий SessionFactory, регистрируя все сущности методом addAnnotatedClass(). Создание фабрики – дорогая операция, поэтому все DAO-классы используют общий экземпляр.

3 Классы-модели и отношения между таблицами

Классы-модели помечены аннотациями @Entity и @Table, поля – аннотациями @Column. Первичные ключи генерируются базой данных (@GeneratedValue(strategy = GenerationType.IDENTITY)). Перечисления Role, TaskStatus и BidStatus хранятся в виде строк благодаря аннотации @Enumerated(EnumType.STRING). Между сущностями настроены следующие связи:

– Task – Bid: двусторонняя связь один ко многим. В классе Bid поле task помечено @ManyToOne и @JoinColumn, в классе Task список bids помечен @OneToMany(mappedBy = "task") с параметрами cascade = CascadeType.ALL и orphanRemoval = true, поэтому при удалении задачи удаляются и все её отклики;

– Category – Task: один ко многим без каскада – задачи не должны пропадать вместе с категорией;

– Category – Category: связь таблицы с самой собой через поле parent (родительская категория);

– AppUser – ExecutorProfile: один к одному. В профиле поле user помечено @OneToOne и @MapsId, поэтому первичный ключ профиля совпадает с идентификатором пользователя;

– AppUser – Task и AppUser – Bid: многие к одному (заказчик задачи и автор отклика).

Для связей @ManyToOne выбран тип извлечения EAGER: заказчик, категория и исполнитель почти всегда выводятся вместе с задачей или откликом. Для коллекций @OneToMany выбран LAZY: список откликов или задач может быть большим и загружается только по необходимости.

Класс Task (геттеры и сеттеры, а также методы equals() и hashCode() опущены):



```java
package by.khaltura.lab4.model;

import ...

@Entity
@Table(name = "task")
public class Task {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "task_id")
    private Integer taskId;

    @ManyToOne(fetch = FetchType.EAGER)  3 usages
    @JoinColumn(name = "customer_id", nullable = false)
    private AppUser customer;

    @ManyToOne(fetch = FetchType.EAGER)  4 usages
    @JoinColumn(name = "category_id", nullable = false)
    private Category category;

    @Column(name = "title", nullable = false)  4 usages
    private String title;

    @Column(name = "description", nullable = false, columnDefinition = "TEXT")  3 usages
    private String description;

    @Column(name = "budget", nullable = false, precision = 10, scale = 2)  4 usages
    private BigDecimal budget;

    @Column(name = "deadline", nullable = false)  4 usages
    private LocalDate deadline;

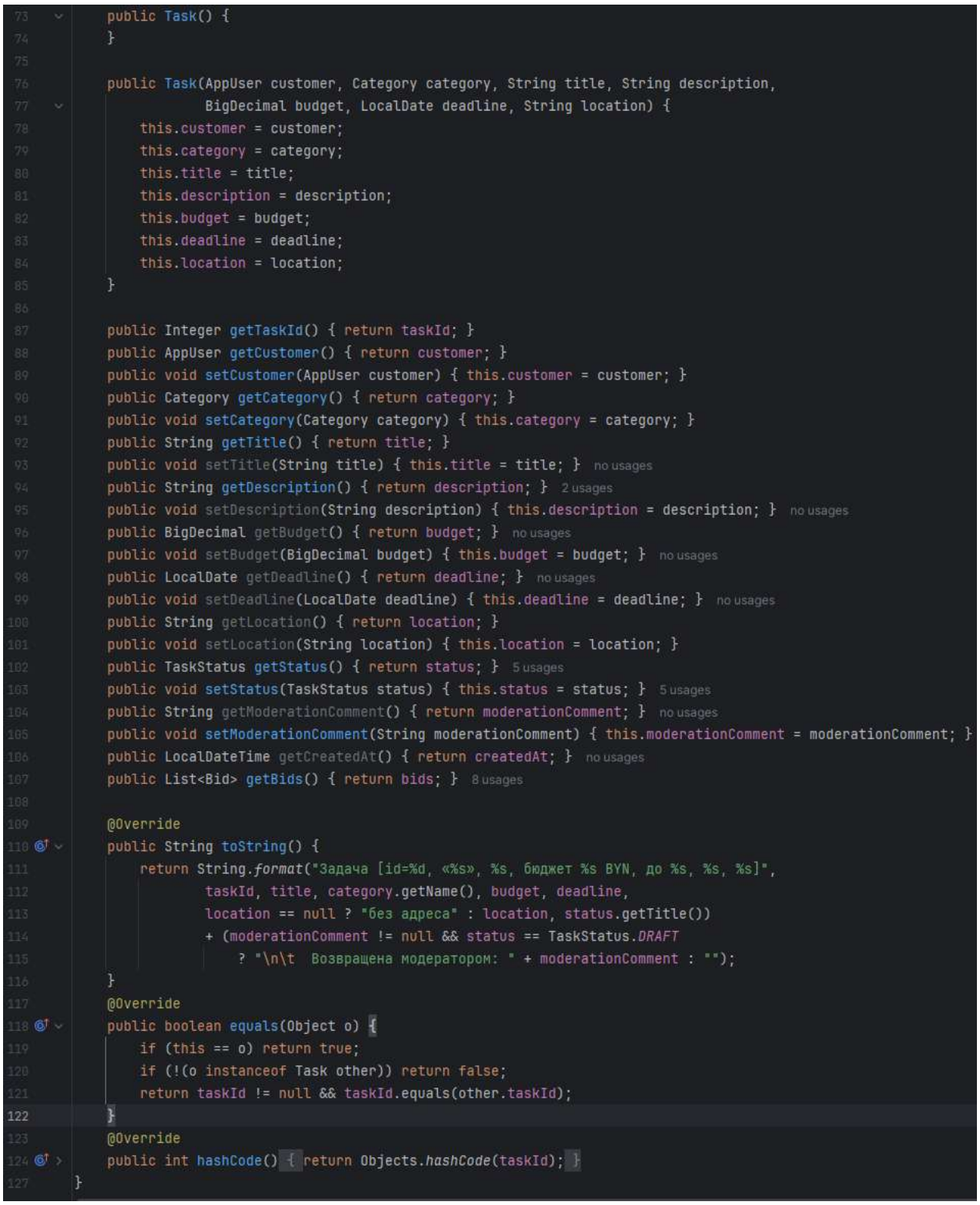
    @Column(name = "location")  5 usages
    private String location;

    @Enumerated(EnumType.STRING)  4 usages
    @Column(name = "status", nullable = false, length = 20)
    private TaskStatus status = TaskStatus.DRAFT;

    @Column(name = "moderation_comment", columnDefinition = "TEXT")  4 usages
    private String moderationComment;

    @Column(name = "created_at", nullable = false,  1 usage
            insertable = false, updatable = false)
    private LocalDateTime createdAt;

    @OneToMany(mappedBy = "task", cascade = CascadeType.ALL,  1 usage
               orphanRemoval = true, fetch = FetchType.LAZY)
    @OrderBy("proposedPrice ASC")
    private List<Bid> bids = new ArrayList<>();
```



```java
    public Task() {
    }

    public Task(AppUser customer, Category category, String title, String description,
                BigDecimal budget, LocalDate deadline, String location) {
        this.customer = customer;
        this.category = category;
        this.title = title;
        this.description = description;
        this.budget = budget;
        this.deadline = deadline;
        this.location = location;
    }

    public Integer getTaskId() { return taskId; }
    public AppUser getCustomer() { return customer; }
    public void setCustomer(AppUser customer) { this.customer = customer; }
    public Category getCategory() { return category; }
    public void setCategory(Category category) { this.category = category; }
    public String getTitle() { return title; }
    public void setTitle(String title) { this.title = title; } no usages
    public String getDescription() { return description; } 2 usages
    public void setDescription(String description) { this.description = description; } no usages
    public BigDecimal getBudget() { return budget; } no usages
    public void setBudget(BigDecimal budget) { this.budget = budget; } no usages
    public LocalDate getDeadline() { return deadline; } no usages
    public void setDeadline(LocalDate deadline) { this.deadline = deadline; } no usages
    public String getLocation() { return location; }
    public void setLocation(String location) { this.location = location; }
    public TaskStatus getStatus() { return status; } 5 usages
    public void setStatus(TaskStatus status) { this.status = status; } 5 usages
    public String getModerationComment() { return moderationComment; } no usages
    public void setModerationComment(String moderationComment) { this.moderationComment = moderationComment; }
    public LocalDateTime getCreatedAt() { return createdAt; } no usages
    public List<Bid> getBids() { return bids; } 8 usages

    @Override
    public String toString() {
        return String.format("Задача [id=%d, «%s», %s, бюджет %s BYN, до %s, %s, %s]",
                taskId, title, category.getName(), budget, deadline,
                location == null ? "без адреса" : location, status.getTitle())
                + (moderationComment != null && status == TaskStatus.DRAFT
                    ? "\n\t  Возвращена модератором: " + moderationComment : "");
    }
    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Task other)) return false;
        return taskId != null && taskId.equals(other.taskId);
    }
    @Override
    public int hashCode() { return Objects.hashCode(taskId); }
}
```

4 Классы DAO

Для каждой сущности создан DAO-класс: UserDAO, CategoryDAO, TaskDAO и BidDAO. Каждый метод получает текущую сессию, открывает транзакцию и фиксирует её методом commit(). В отличие от методического примера, при исключении транзакция откатывается методом rollback().

Отдельный вызов session.close() не нужен: сессия, привязанная к потоку, закрывается автоматически после commit() или rollback().

По заданию 1 реализованы методы, помеченные в методическом примере комментарием «ЗАДАНИЕ»:

– findCategoryById() и findTaskById() – поиск по идентификатору (аналог findDepartmentByID);

– addCategory() и addTask() – добавление записи методом persist() (аналог addDepartament);

– moveTaskToCategory() – перенос задачи в другую категорию (аналог updateDevelopersDepartment). Задача загружается в сессию, ей назначается новая категория, а изменение сохраняется при фиксации транзакции благодаря механизму отслеживания изменений (dirty checking);

– deleteTask() – удаление задачи вместе с откликами за счёт каскада (аналог deleteDepartment);

– getUserByEmail() – поиск пользователя по email запросом HQL с параметром (аналог getUserByUsername).

По заданию 2 в классе TaskDAO написаны три собственных SELECT-запроса:

– findPublishedByBudget() (HQL) – опубликованные задачи с бюджетом в заданном диапазоне, отсортированные по бюджету. Если указана родительская категория, в выборку попадают и задачи её дочерних категорий: для этого используется соединение с категорией и левое соединение с родительской категорией;

– findByCityAndStatus() (Criteria) – поиск задач по подстроке адреса без учёта регистра. Условие по статусу добавляется в список предикатов, только если статус указан, что показывает удобство Criteria API для динамически формируемых запросов;

– categoryStats() (HQL) – запрос с соединением, группировкой и агрегатными функциями COUNT, AVG, MIN и MAX: число задач, средний, минимальный и максимальный бюджет по каждой категории без учёта черновиков и отменённых задач. Результат преобразуется в записи (record) CategoryStats.

Во всех запросах значения передаются через setParameter(), а не подставляются в текст запроса, что исключает SQL-инъекции. Фрагмент класса TaskDAO с методами задания 1 для задач и тремя собственными запросами:

```java
package by.khaltura.lab4.dao;

import ...

public class TaskDAO {  18 usages

    private final SessionConfig sc = SessionConfig.getInstance();  14 usages

    public record CategoryStats(String category, long tasks, BigDecimal avgBudget,  8 usages
                                BigDecimal minBudget, BigDecimal maxBudget) {  1 usage
    }

    public Task findTaskById(Integer id) {  4 usages
        Session session = sc.getSessionFactory().getCurrentSession();
        Transaction tx = session.beginTransaction();
        try {
            Task task = session.get(Task.class, id);
            tx.commit();
            return task;
        } catch (RuntimeException e) {
            tx.rollback();
            throw e;
        }
    }

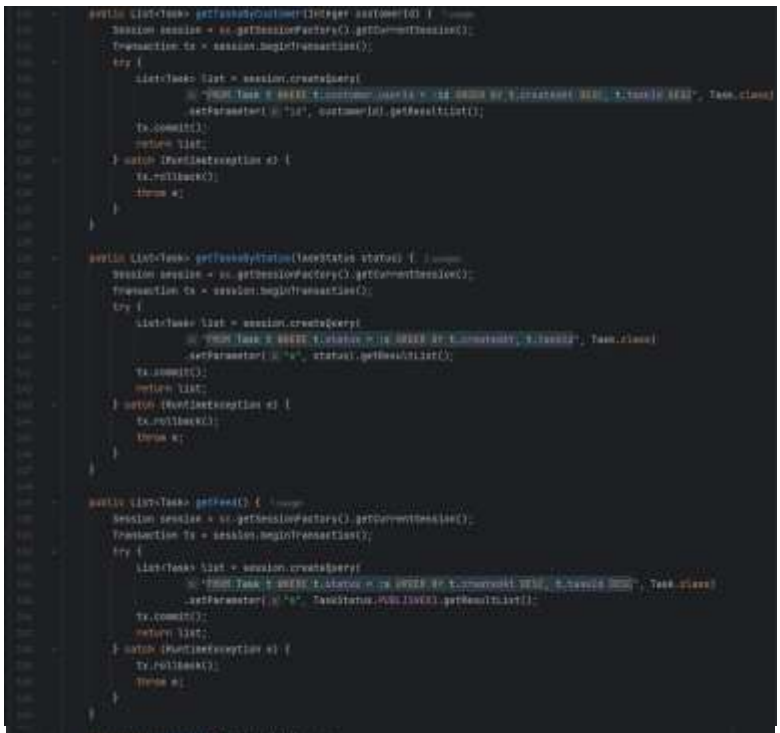
    public Task getTaskWithBids(Integer id) {  5 usages
        Session session = sc.getSessionFactory().getCurrentSession();
        Transaction tx = session.beginTransaction();
        try {
            Task task = session.get(Task.class, id);
            if (task != null) {
                Hibernate.initialize(task.getBids());
            }
            tx.commit();
            return task;
        } catch (RuntimeException e) {
            tx.rollback();
            throw e;
        }
    }

    public Task addTask(Task task) {  1 usage
        Session session = sc.getSessionFactory().getCurrentSession();
        Transaction tx = session.beginTransaction();
        try {
            session.persist(task);
            tx.commit();
            return task;
        } catch (RuntimeException e) {
            tx.rollback();
            throw e;
        }
    }
```

```java
    public Task updateTask(Task task) {  3 usages
        Session session = sc.getSessionFactory().getCurrentSession();
        Transaction tx = session.beginTransaction();
        try {
            Task merged = session.merge(task);
            tx.commit();
            return merged;
        } catch (RuntimeException e) {
            tx.rollback();
            throw e;
        }
    }

    public Task moveTaskToCategory(Integer taskId, Category category) {  1 usage
        Session session = sc.getSessionFactory().getCurrentSession();
        Transaction tx = session.beginTransaction();
        try {
            Task task = session.get(Task.class, taskId);
            if (task == null) {
                throw new IllegalArgumentException("Задача не найдена");
            }
            task.setCategory(session.get(Category.class, category.getCategoryId()));
            tx.commit();
            return task;
        } catch (RuntimeException e) {
            tx.rollback();
            throw e;
        }
    }

    public void deleteTask(Integer id) {  1 usage
        Session session = sc.getSessionFactory().getCurrentSession();
        Transaction tx = session.beginTransaction();
        try {
            Task task = session.get(Task.class, id);
            if (task == null) {
                throw new IllegalArgumentException("Задача не найдена");
            }
            session.remove(task);
            tx.commit();
        } catch (RuntimeException e) {
            tx.rollback();
            throw e;
        }
    }
```



```
    public List<Task> getTasksByCustomer(Integer customerId) {
        Session session = sf.getSessionFactory().getCurrentSession();
        Transaction tx = session.beginTransaction();
        try {
            List<Task> list = session.createQuery(
                    "FROM Task t WHERE t.customer.userId = :id ORDER BY t.createdAt DESC, t.taskId DESC", Task.class)
                    .setParameter("id", customerId).getResultList();
            tx.commit();
            return list;
        } catch (RuntimeException e) {
            tx.rollback();
            throw e;
        }
    }

    public List<Task> getTasksByStatus(TaskStatus status) {
        Session session = sf.getSessionFactory().getCurrentSession();
        Transaction tx = session.beginTransaction();
        try {
            List<Task> list = session.createQuery(
                    "FROM Task t WHERE t.status = :s ORDER BY t.createdAt, t.taskId",
                    Task.class)
                    .setParameter("s", status).getResultList();
            tx.commit();
            return list;
        } catch (RuntimeException e) {
            tx.rollback();
            throw e;
        }
    }

    public List<Task> getFeed() {
        Session session = sf.getSessionFactory().getCurrentSession();
        Transaction tx = session.beginTransaction();
        try {
            List<Task> list = session.createQuery(
                    "FROM Task t WHERE t.status = :s ORDER BY t.createdAt DESC, t.taskId DESC", Task.class)
                    .setParameter("s", TaskStatus.PUBLISHED).getResultList();
            tx.commit();
            return list;
        } catch (RuntimeException e) {
            tx.rollback();
            throw e;
        }
    }

    public void acceptBid(Integer bidId) {
        Session session = sf.getSessionFactory().getCurrentSession();
        Transaction tx = session.beginTransaction();
        try {
            Bid bid = session.get(Bid.class, bidId);
            Task task = bid.getTask();
            bid.setStatus(BidStatus.ACCEPTED);
            session.createMutationQuery(
                    "UPDATE Bid b SET b.status = :rejected WHERE b.task = :task AND b.bidId <> :id AND b.status = :pending")
                    .setParameter("rejected", BidStatus.REJECTED)
                    .setParameter("task", task)
                    .setParameter("id", bidId)
                    .setParameter("pending", BidStatus.PENDING)
                    .executeUpdate();
            task.setStatus(TaskStatus.IN_PROGRESS);
            tx.commit();
        } catch (RuntimeException e) {
            tx.rollback();
            throw e;
        }
    }

    public void cancelTask(Integer taskId) {
        Session session = sf.getSessionFactory().getCurrentSession();
        Transaction tx = session.beginTransaction();
        try {
            Task task = session.get(Task.class, taskId);
            task.setStatus(TaskStatus.CANCELLED);
            session.createMutationQuery("UPDATE Bid b SET b.status = :r WHERE b.task = :t AND b.status = :p")
                    .setParameter("r", BidStatus.REJECTED)
                    .setParameter("t", task)
                    .setParameter("p", BidStatus.PENDING)
                    .executeUpdate();
            tx.commit();
        } catch (RuntimeException e) {
            tx.rollback();
            throw e;
        }
    }

    public List<Task> findAffordableByBudget(BigDecimal min, BigDecimal max,
                                             Integer categoryId) {
        Session session = sf.getSessionFactory().getCurrentSession();
        Transaction tx = session.beginTransaction();
        try {
            // LEFT JOIN + COALESCE: a customer with no budget set is treated as $0 max
            String hql = "SELECT t FROM Task t JOIN t.category c LEFT JOIN t.owner o"
                    + " WHERE t.status = :status AND t.budget BETWEEN :min AND :max"
                    + (categoryId != null
                            ? " AND (c.categoryId = :cat OR c.parentId = :cat)" : "")
                    + " ORDER BY t.budget";
            var query = session.createQuery(hql, Task.class)
                    .setParameter("status", TaskStatus.PUBLISHED)
                    .setParameter("min", min)
                    .setParameter("max", max);
            if (categoryId != null) {
                query.setParameter("cat", categoryId);
            }
            List<Task> list = query.getResultList();
            tx.commit();
            return list;
        } catch (RuntimeException e) {
            tx.rollback();
            throw e;
        }
    }
```



```
    public List<Task> findByCityAndStatus(String city, TaskStatus status) {  1 usage
        Session session = sc.getSessionFactory().getCurrentSession();
        Transaction tx = session.beginTransaction();
        try {
            CriteriaBuilder cb = session.getCriteriaBuilder();
            CriteriaQuery<Task> criteria = cb.createQuery(Task.class);
            Root<Task> root = criteria.from(Task.class);
            List<Predicate> where = new ArrayList<>();
            String pattern = "%" + city.trim().toLowerCase() + "%";
            where.add(cb.like(cb.lower(root.get("location")), pattern));
            if (status != null) {
                where.add(cb.equal(root.get("status"), status));
            }
            criteria.select(root).where(cb.and(where.toArray(new Predicate[0])))
                    .orderBy(cb.desc(root.get("createdAt")));
            List<Task> list = session.createQuery(criteria).getResultList();
            tx.commit();
            return list;
        } catch (RuntimeException e) {
            tx.rollback();
            throw e;
        }
    }

    public List<CategoryStats> categoryStats() {  1 usage
        Session session = sc.getSessionFactory().getCurrentSession();
        Transaction tx = session.beginTransaction();
        try {
            List<Object[]> rows = session.createQuery(
                    s: "SELECT c.name, COUNT(t), AVG(t.budget), "
                            + "MIN(t.budget), MAX(t.budget) "
                            + "FROM Task t JOIN t.category c "
                            + "WHERE t.status NOT IN (:draft, :cancelled) "
                            + "GROUP BY c.name ORDER BY COUNT(t) DESC, c.name",
                    Object[].class)
                    .setParameter( s: "draft", TaskStatus.DRAFT)
                    .setParameter( s: "cancelled", TaskStatus.CANCELLED)
                    .getResultList();
            tx.commit();
            List<CategoryStats> result = new ArrayList<>();
            for (Object[] r : rows) {
                result.add(new CategoryStats((String) r[0], (Long) r[1],
                        BigDecimal.valueOf((Double) r[2])
                                .setScale( newScale: 2, RoundingMode.HALF_UP),
                        (BigDecimal) r[3], (BigDecimal) r[4]));
            }
            return result;
        } catch (RuntimeException e) {
            tx.rollback();
            throw e;
        }
    }
}
```

5 Проблема n+1 и ленивая загрузка

При выводе всех категорий вместе с задачами без специальных мер Hibernate выполнил бы один запрос за категориями и по отдельному запросу на задачи каждой категории. Метод CategoryDAO.getCategoriesWithTasks() решает эту проблему запросом «FROM Category c LEFT JOIN FETCH c.tasks t LEFT JOIN FETCH t.customer»: категории, их задачи и заказчики загружаются одним SQL-запросом. Так как соединение повторяет категорию для каждой её задачи, результат складывается в множество LinkedHashSet.

Второй способ показан в методе TaskDAO.getTaskWithBids(): ленивый список откликов инициализируется вызовом Hibernate.initialize() внутри открытой сессии. Без этого обращение к списку после закрытия сессии привело бы к исключению LazyInitializationException. По этой же причине в методах toString() не используются ленивые поля.

6 Бизнес-логика и консольное меню

Правила предметной области вынесены в классы TaskService и BidService:

– задача проходит состояния «черновик → на модерации → опубликована → в работе → завершена», недопустимые переходы запрещены методом TaskStatus.canMoveTo();

– администратор публикует задачу или возвращает её в черновик с обязательным указанием причины;

– создавать задачи и откликаться может только пользователь с подтверждённой личностью;

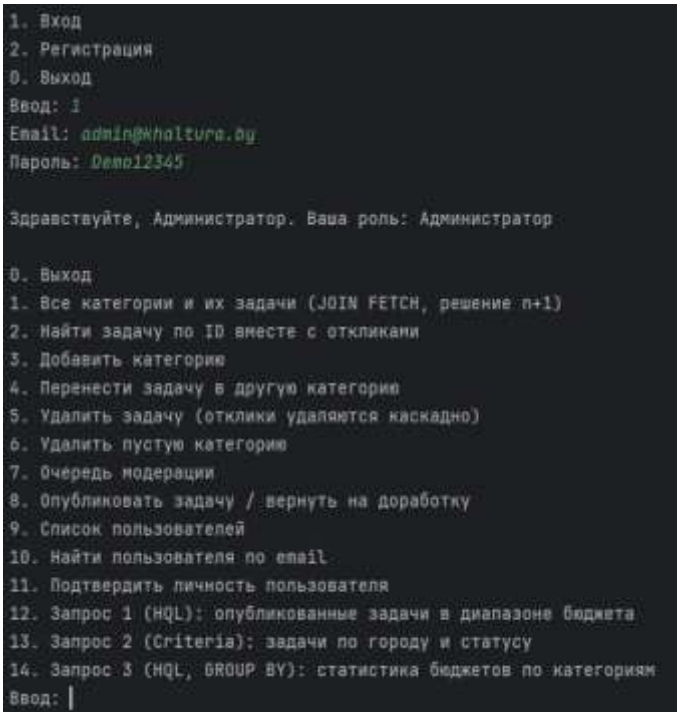
– нельзя откликнуться на собственную задачу или на неопубликованную задачу, отозванный отклик можно подать заново;

– при выборе исполнителя в одной транзакции выбранный отклик принимается, остальные ожидающие отклоняются, а задача переходит в работу.

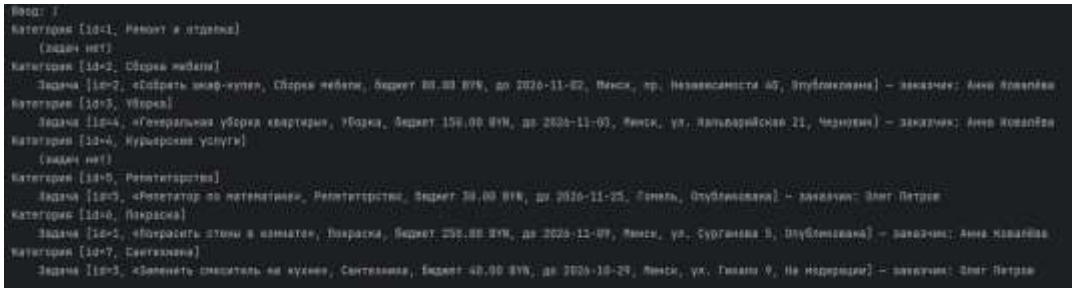
Класс MainClass выполняет вход и регистрацию и в зависимости от роли открывает меню AdminMenu, CustomerMenu или ExecutorMenu. Пользователь с ролью «заказчик и исполнитель» при входе выбирает режим работы. Пункты меню администратора 2–5 и 10 соответствуют заданию 1, пункты 12–14 – заданию 2.

7 Проверка работы приложения

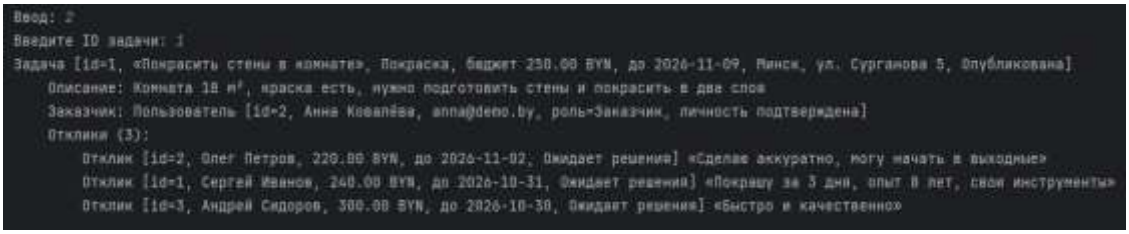
Вход администратора (admin@khaltura.by, пароль Demo12345) и меню администратора:



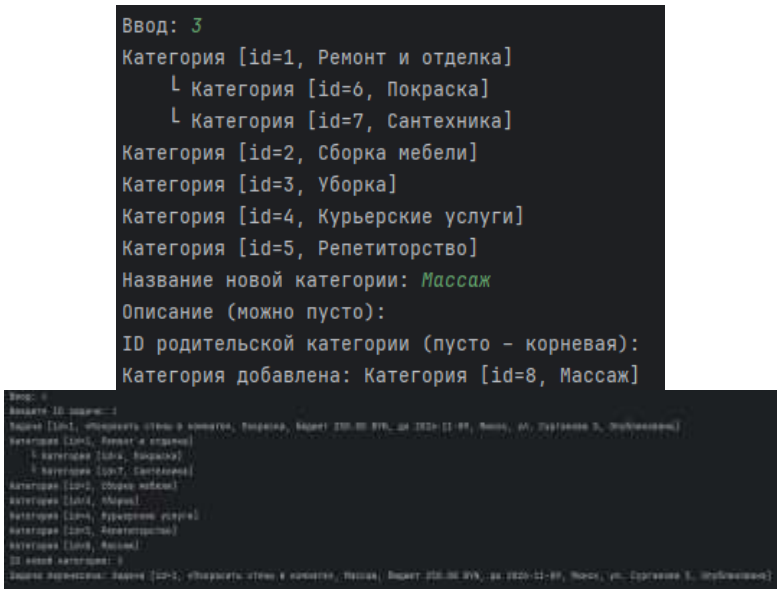
Пункт 1 – все категории с задачами, загруженные одним запросом с JOIN FETCH:



Пункт 2 – поиск задачи с идентификатором 1 вместе с откликами:



Пункты 3 и 4 – добавление дочерней категории и перенос задачи в другую категорию:



Пункт 5 – удаление задачи: вместе с ней каскадно удаляются её отклики:



Пункт 10 – поиск пользователя по email:

```
Ввод: 10
Введите email: anna@demo.by
Пользователь [id=2, Анна Ковалёва, anna@demo.by, роль=Заказчик, личность подтверждена]
```

Пункт 12 – запрос HQL: опубликованные задачи в диапазоне бюджета:

```
Ввод: 12
Бюджет от: 100
Бюджет до: 300
Категория [id=1, Ремонт и отделка]
    └ Категория [id=6, Покраска]
    └ Категория [id=7, Сантехника]
Категория [id=2, Сборка мебели]
Категория [id=3, Уборка]
Категория [id=4, Курьерские услуги]
Категория [id=5, Репетиторство]
Категория [id=8, Массаж]
ID категории (пусто - все; родительская включает дочерние): 1
Ничего не найдено.
```

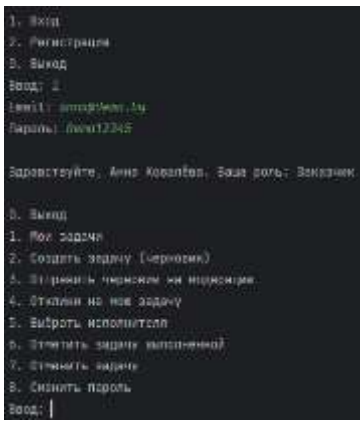
Пункт 13 – запрос Criteria: задачи по городу и статусу:

```
Ввод: 13
Город (или часть адреса): Минск
Статусы: [DRAFT, PENDING_MODERATION, PUBLISHED, IN_PROGRESS, COMPLETED, CANCELLED]
Статус (пусто - любой):
Задача [id=2, «Собрать шкаф-купе», Сборка мебели, бюджет 80.00 BYN, до 2026-11-02, Минск, пр. Независимости 40, Опубликована]
Задача [id=3, «Заменить смеситель на кухне», Сантехника, бюджет 40.00 BYN, до 2026-10-29, Минск, ул. Гикало 9, На модерации]
Задача [id=4, «Генеральная уборка квартиры», Уборка, бюджет 150.00 BYN, до 2026-11-05, Минск, ул. Кальварийская 21, Черновик]
```

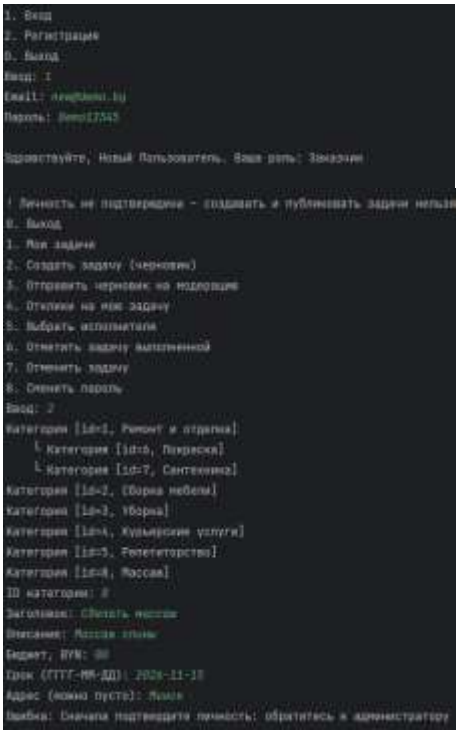
Пункт 14 – запрос HQL с группировкой: статистика бюджетов по категориям:

```
Ввод: 14
Категория                Задач     Средний         Мин         Макс
Репетиторство                1       30.00       30.00       30.00
Сантехника                   1       40.00       40.00       40.00
Сборка мебели                1       80.00       80.00       80.00
```

Вход заказчика (anna@demo.by): просмотр откликов на задачу и выбор исполнителя, остальные отклики отклонены:



Пользователь без подтверждённой личности (new@demo.by) не может создать задачу:



Вывод: В результате выполнения работы получены практические навыки работы с JPA и её реализацией Hibernate. Классы предметной области «Халтура» отображены на таблицы PostgreSQL с помощью аннотаций, настроены связи один ко многим, многие к одному, один к одному с общим первичным ключом и связь таблицы с самой собой, изучены каскадные операции и типы извлечения данных LAZY и EAGER. Реализованы методы, указанные в задании, и три собственных запроса: два на языке HQL, в том числе с группировкой и агрегатными функциями, и один с помощью JPA Criteria API. Решена проблема n+1 с помощью JOIN FETCH, рассмотрена инициализация ленивых коллекций и обработка транзакций с откатом при ошибке.